How Java Compiles & Runs

The principle behind the two commands — and the role of the JDBC driver JAR

Bank Management System · Java 17 · MySQL · MySQL Connector/J 8.3.0

1. The Big Picture

Java runs in **two separate steps**. The code you write is **plain text source** (`.java`) that the computer cannot execute directly. First it is **compiled** into **bytecode** (`.class`), then the bytecode is **executed** by the Java Virtual Machine (JVM).

```
App.java ──(javac: compiler)──▶ App.class / bytecode ──(java: JVM)──▶ program runs source code machine-readable application
```

This is why there are **two commands**: one to **compile**, one to **run**. They do different jobs and you generally compile once, then run as many times as you like.

2. Step 1 — Compile

```
javac -cp "lib\mysql-connector-j-8.3.0.jar" -d bin -sourcepath src src\App.java
```

Piece	What it does
`javac`	The Java **compiler**. Reads source files and produces bytecode `.class` files (in the default package here).
`-cp "lib\...\.jar"`	**Class-path** — the list of places the compiler looks for *already-compiled* classes it needs. Here it points at the MySQL driver JAR.
`-d bin`	**Destination** — the folder where all compiled `.class` files are written.
`-sourcepath src`	Where to look for the **other source files** (`ui\LoginFrame.java` , `db\DatabaseOperations.java` , ...).
`src\App.java`	The **starting file**. The compiler follows its references and compiles the rest automatically.

Compilation only needs the driver JAR because the source refers to MySQL/JDBC classes; the rest of *our* code is found through `-sourcepath` .

3. Step 2 — Run

```
java -cp "bin;lib\mysql-connector-j-8.3.0.jar" App
```

Piece	What it does
`java`	Starts the **JVM**, which loads and executes the compiled bytecode.
`-cp "bin;lib\...\.jar"`	**Class-path** at run time: `bin` (our compiled classes) **and** the driver JAR. On Windows the separator between entries is a semicolon `;` .
`App`	The **main class** to start (write `App` , not `App.class`).

4. The Class-path — the connecting thread

The **class-path** is simply *"where should Java look for the classes I use?"* It appears in **both** commands, for the same reason:

- **At compile time** the compiler needs to know the types you reference, so it can check your code and emit bytecode.
- **At run time** the JVM needs to actually load those same classes to execute the program.

> **Why it matters:** our own `App` lives in `bin`; the MySQL driver lives in the JAR. If either is missing from the class-path at the right moment, the program cannot find it. That is exactly why the two commands look so similar — both must list the driver JAR.

5. What is the JAR, and where does it come from?

> A **JAR** (*Java ARchive*) is a single file that bundles many compiled Java `.class` files and resources together — essentially a `.zip` for Java. It is a **library**: pre-written, pre-compiled code you reuse instead of writing yourself.

`lib\mysql-connector-j-8.3.0.jar` is the **MySQL Connector/J** driver. It is **not part of the JDK** and is **not written by us** — it is a separate product that is **downloaded from the internet** (from the official MySQL download site, `dev.mysql.com/downloads/connector/j`) and then placed in this project's `lib` folder. The version number `8.3.0` is the release of the driver.

6. The Role of the JDBC Driver

Java does not know how to talk to MySQL by itself. JDBC (*Java Database Connectivity*) is only an **interface** — a standard set of rules and types (`Connection`, `Statement`, `ResultSet`...) that all databases agree to follow. It is deliberately database-agnostic.

For a specific database — MySQL — you need a concrete implementation of that interface: the **driver**. That is precisely what the JAR provides. It is the **bridge** between your Java code and the MySQL server:

```
Our code (java.sql.*)
| calls JDBC methods, e.g. DriverManager.getConnection(...)
▼
MySQL Connector/J JAR ◀── the driver: implements the JDBC interfaces
| translates those calls into the MySQL network protocol,
| e.g. connects to localhost:3306 and speaks MySQL's language
▼
MySQL Server (bank_db) ── executes SQL, returns rows ──▶ back up the chain
```

In short, the driver JAR:

- provides the class `com.mysql.cj.jdbc.Driver` — the MySQL implementation of JDBC;
- lets `DriverManager.getConnection("jdbc:mysql://localhost:3306/bank_db", ...)` actually work;
- converts Java JDBC calls into MySQL's wire protocol and converts results back into Java objects;
- is registered/loaded automatically once it is on the class-path.

> **If the JAR is missing from the run-time class-path**, the program fails with `No suitable driver found for jdbc:mysql://localhost:3306/bank_db`. The code is fine — it simply cannot find any driver that understands `jdbc:mysql://`.

7. Pulling it all together

```
1. You write → src\App.java (+ ui\, db\, classes\ ...)
2. Compile → javac ... → bin\App.class (bytecode)
3. Run → java ... App → JVM starts the program
4. Connect to MySQL → the driver JAR translates JDBC calls to MySQL
5. Data round-trip → MySQL server (localhost:3306) ⇔ your application
```

8. Quick reference

Thing	Meaning in one line
`javac`	Compiles `.java` source into `.class` bytecode.
`java`	Starts the JVM and runs a compiled main class.
`-cp` / `-classpath`	Where Java looks for classes (our `bin` + the driver JAR).
`bin`	Folder holding our compiled bytecode.
`-sourcepath src`	Where the compiler finds the other source files.
`.jar`	A bundled Java library — here, the downloaded MySQL driver.
JDBC	The Java interface for databases; the driver is its MySQL implementation.